AI Engineering and Claude Architect · Part III, Agents

Agents and MCP, Lesson 2

The Agentic Loop

The agent wrote "Done reading your message, checking the order now." The harness saw the word "done" and ended the run. The customer got half a sentence, and the refund never happened.

Module	AI06 of 13
Lesson	2 of 10
Prerequisites	AI03 L1, AI03 L5, AI06 L1
Exam relevance	**Direct.** All of TS 1.1
Reading time	~40 minutes

Contents

1. The Agentic Loop

Lesson 2 of 10 · Agents and MCP

The Agentic Loop

Key idea

An agent is a loop: send the conversation, run the tools Claude asks for, append the results, and repeat until `stop_reason` says it's finished. Anthropic describes agents as "LLMs using tools based on environmental feedback in a loop." **Control flow reads `stop_reason`, and never the words in the text.** An iteration cap is a safety net that reports failure, not the normal way to stop.

> *Before: statelessness and stop reasons (AI03 L1), caching (AI03 L5). Teaches: the loop's lifecycle, parallel tool calls, every stop reason, anti-patterns, cost growth, tool runner vs manual loop. Used by: all of AI06, AI08 L3, AI12.*

The hook: three ways to stop

Here's the same four-reply conversation, stopped three ways. It's scripted, so no model is involved, and it tests only the harness. Real output from `code/ai06-agents/.../AgentLoop.java`, with line breaks added:

```
Same four-reply script, three ways of stopping:
 stop_reason   Outcome[status=DONE, modelCalls=4, toolRuns=4,
               lastText=Your refund of $40 is on its way.]
 word "done"   Outcome[status=STOPPED_ON_WORD, modelCalls=1, toolRuns=0,
               lastText=Done reading your message. Checking the order.]
 cap of 2      Outcome[status=STOPPED_AT_CAP, modelCalls=2, toolRuns=3,
               lastText=Checking the customer and the refund policy.]
```

Key idea

Only `stop_reason` finished the job. The word check ended before any tool ran. The cap ended mid-task, and it looked like a normal exit. Both fail silently, and they're exactly the anti-patterns **TS 1.1** names.

Intuition: a doctor and a lab

A doctor examines you and orders blood tests. The lab runs them and sends back results. The doctor reads those, and may order more, or may give a diagnosis.

The doctor never leaves the room to run the tests, and the lab never decides what's wrong. **Claude is the doctor. Your code is the lab.** Claude asks, your code does, and the result goes in the chart. The visit ends when the doctor says so, not when the doctor happens to say the word "fine".

Mechanism: the loop

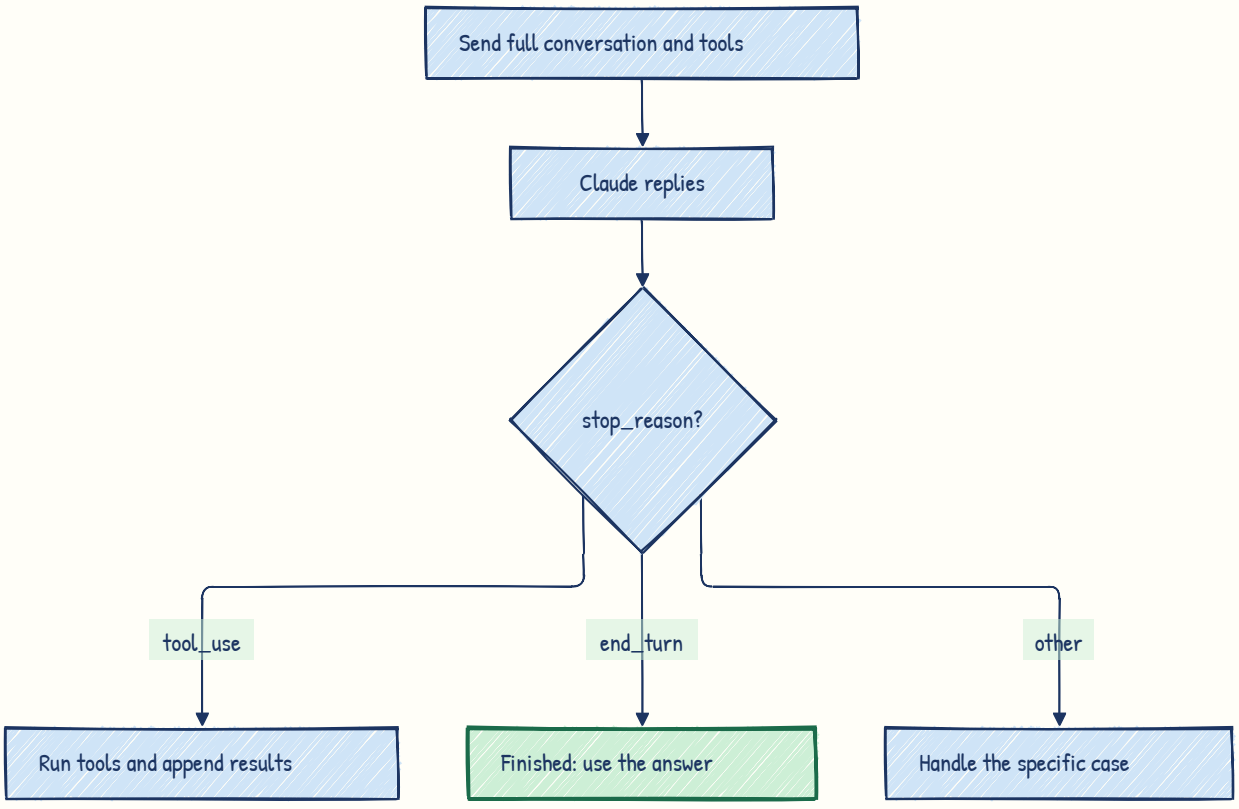


Figure 1. One iteration of the loop. Claude never runs a tool itself. Your code runs it, appends the result, and sends the whole conversation again.

Step by step:

1. Send the conversation, plus tool definitions.
2. Claude responds. A `tool_use` response holds one or more `tool_use` blocks, each with an `id`, a `name` and an `input`.
3. Your code validates and runs each one.
4. Append **the whole assistant message**, then one user message containing all the `tool_result` blocks, each with its matching `tool_use_id`.
5. Send again. The API is stateless, so this contains everything (AI03 L1).

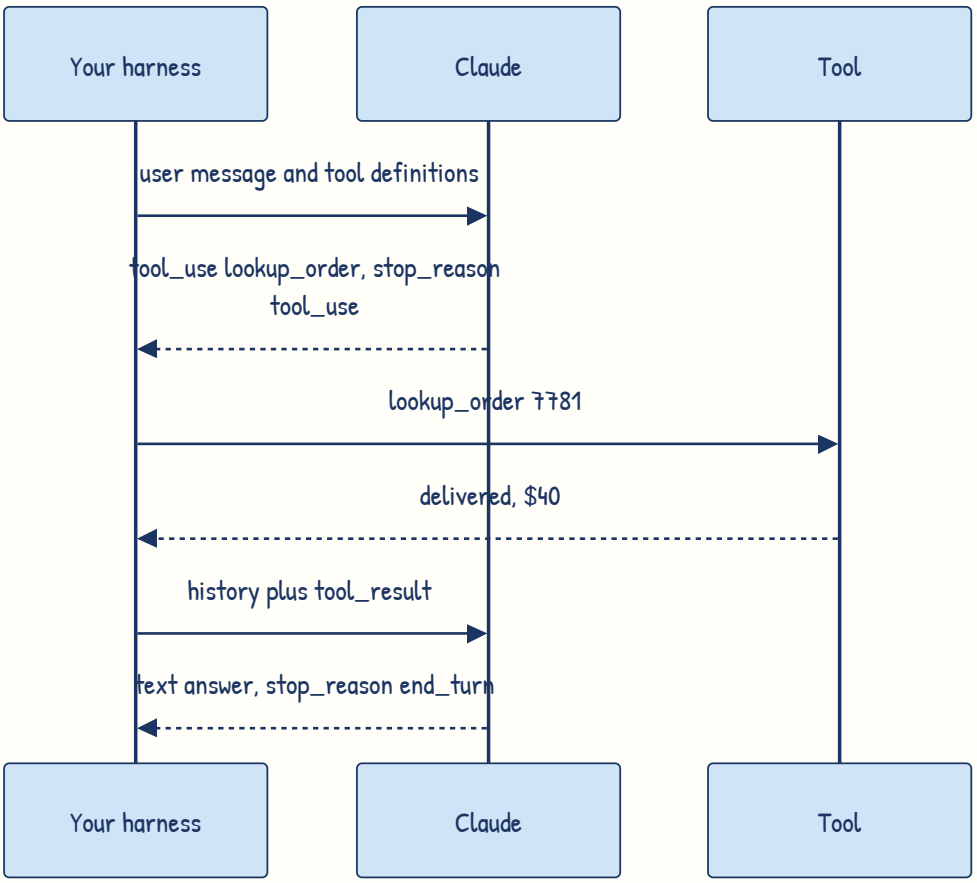


Figure 2. Messages over two iterations. The harness holds all state, and each request carries the entire conversation so far.

Every stop reason

The exam guide focuses on the first two. Production code needs them all.

stop_reason	Meaning	What your loop does
`tool_use`	Claude wants tools run	Run them, append results, continue
`end_turn`	Claude has finished	Return the answer
`pause_turn`	A server-side tool loop paused	Resend as is. **Don't add a "continue" message**
`max_tokens`	Output was cut off	**Never run this turn's tools**, since input may be cut. Raise the limit and retry
`refusal`	A safety classifier declined	Stop, and don't read or run anything. Consider a fallback (AI03 L1)
`stop_sequence`	A custom stop string was hit	Application-specific

Real output for those paths, from the same lab:

```
Other stop reasons:
 max_tokens    Outcome[status=TRUNCATED, modelCalls=1, toolRuns=0, lastText=Issuing]
 refusal       Outcome[status=REFUSED, modelCalls=1, toolRuns=0, lastText=]
 pause_turn    Outcome[status=DONE, modelCalls=2, toolRuns=0, lastText=Found it.]
 stuck model   Outcome[status=SAFETY_LIMIT, modelCalls=25, toolRuns=25, lastText=Trying again]
```

Look at `max_tokens`: a tool call was present, and `toolRuns=0`. A half-written `process_refund` input could parse as a valid but wrong call, so you must not run it.

Parallel tool calls

One response can contain several `tool_use` blocks.

Rule	Why
Run them, concurrently if they're safe to	Lower latency
Return **all** results in **one** user message	Splitting them silently trains Claude out of parallel calls
Return a failed tool as `is_error: true`	Never drop one: every `tool_use` needs a result
Limit to one call with `disable_parallel_tool_use: true`	When order matters

What a tool error looks like

Tool failures don't end the loop. Send the failure back, and Claude can react.

```
{ "type": "tool_result", "tool_use_id": "toolu_01",
  "is_error": true, "content": "order_id must be digits only, such as 7781" }
```

AI06 L5 builds this out into categories.

The loop gets more expensive each turn

Each request resends everything. Illustratively, take a 5,000-token start and 1,000 tokens added per turn, for 10 turns on Opus 5.5 ($4 per million input):

Approach	Input tokens billed	Input cost
No caching	95,000 at full price	**$0.380**
Prompt caching, each turn extending the cache	81,000 reads, 14,000 writes	**$0.086**

Output tokens are extra. So **cache the prefix in any agent** (AI03 L5). For long runs, also trim tool outputs, and use the context tools in AI06 L7.

Why the obvious fixes fall short

Tempting move	Why it fails	Better
Search the text for "done" or "complete"	Natural language is ambiguous. "Done reading" isn't finished	`stop_reason`
Stop when the reply contains text	Claude often writes text **and** calls tools	`stop_reason`
Fixed iteration count as the stopping rule	Cuts off hard tasks, and hides that it did	`end_turn` to finish, with a limit as a **failure alarm**
Save only the text of each reply	Loses `tool_use` blocks, so the next request is invalid	Append the whole message
Retry on every tool error	Some can never succeed	Structured errors (AI06 L5)
Run tools when `max_tokens` hit	Input may be truncated	Raise the limit and retry

A limit is still needed

TS 1.1 warns against arbitrary caps as the **primary** stopping mechanism. A limit as a safety net is sensible. Anthropic's own post lists stopping conditions such as a maximum iteration count as a way to maintain control. The difference is what happens at the limit: **reaching it should report failure or a partial result, not pretend to be a normal finish.**

How Anthropic solved it

Problem	Anthropic's approach	Source
Everyone rewrites the same loop	**Tool Runner** in the SDKs (beta)	Claude API docs
Need the whole harness	**Claude Agent SDK**: Claude Code's loop, tools and hooks as a library	Agent SDK docs
Need hosting as well	**Managed Agents**: Anthropic runs the loop and a per-session sandbox	Claude API docs
Intermediate results flood context	Programmatic tool calling (AI06 L3)	Advanced tool use

Four ways to run the loop

#	Approach	You write	You host	Choose it when
1	Manual loop	The whole loop	Yes	You need control the runner doesn't allow, or no beta dependency
2	Tool Runner	Tool functions	Yes	**Most custom-tool agents**
3	Managed Agents	Config and your tool results	No	You want the loop and a sandbox hosted. It's in beta
4	Agent SDK	Prompt and options	Yes	You want built-in file and bash tools, and hooks

Tool Runner and the Agent SDK sound alike, but they're different. The runner loops over tools you define. The Agent SDK is the full Claude Code harness. The exam's Scenario 4 uses the Agent SDK. Approval gates don't require a manual loop, since a runner can gate inside your tool function.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: loop in the request handler. It's simple. A long run ties up a request, and a crash loses everything.

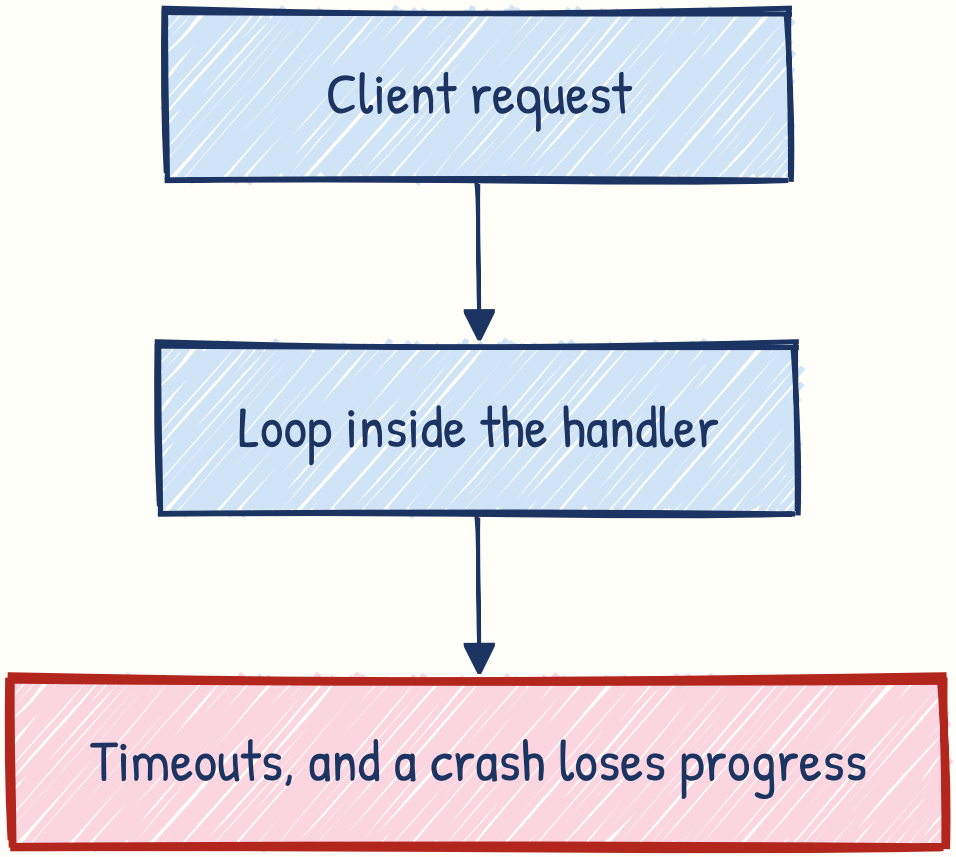


Figure 3. Baseline. The loop lives inside one HTTP request, so a crash or timeout loses all progress.

Improved: a background worker. Long runs survive the request. New problem: **a crash still loses state, and retrying can repeat side effects such as a refund.**

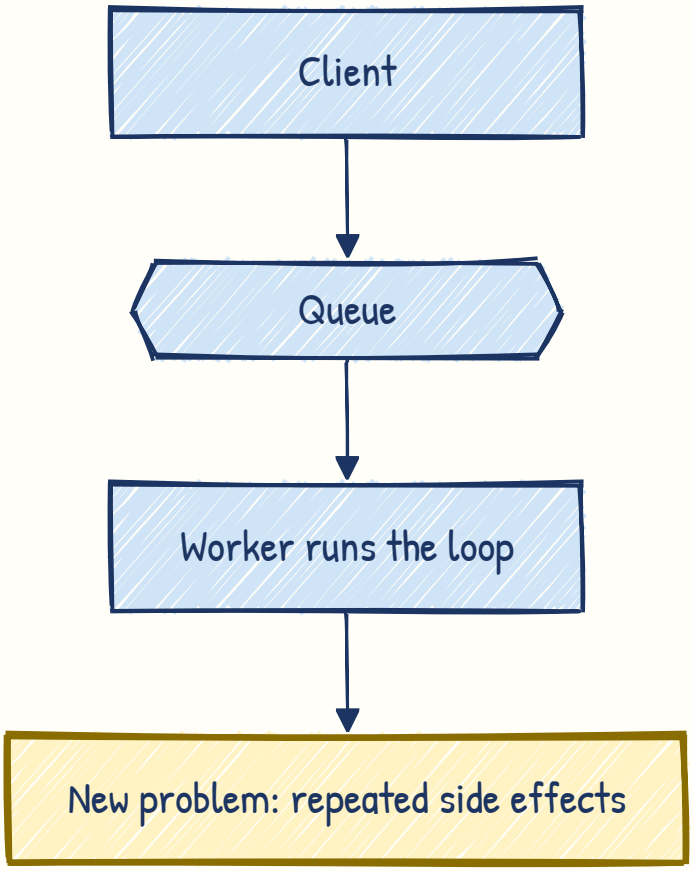


Figure 4. Improved. A queue and worker free the request, but a retried run can issue the same refund twice.

Production: durable, idempotent steps.

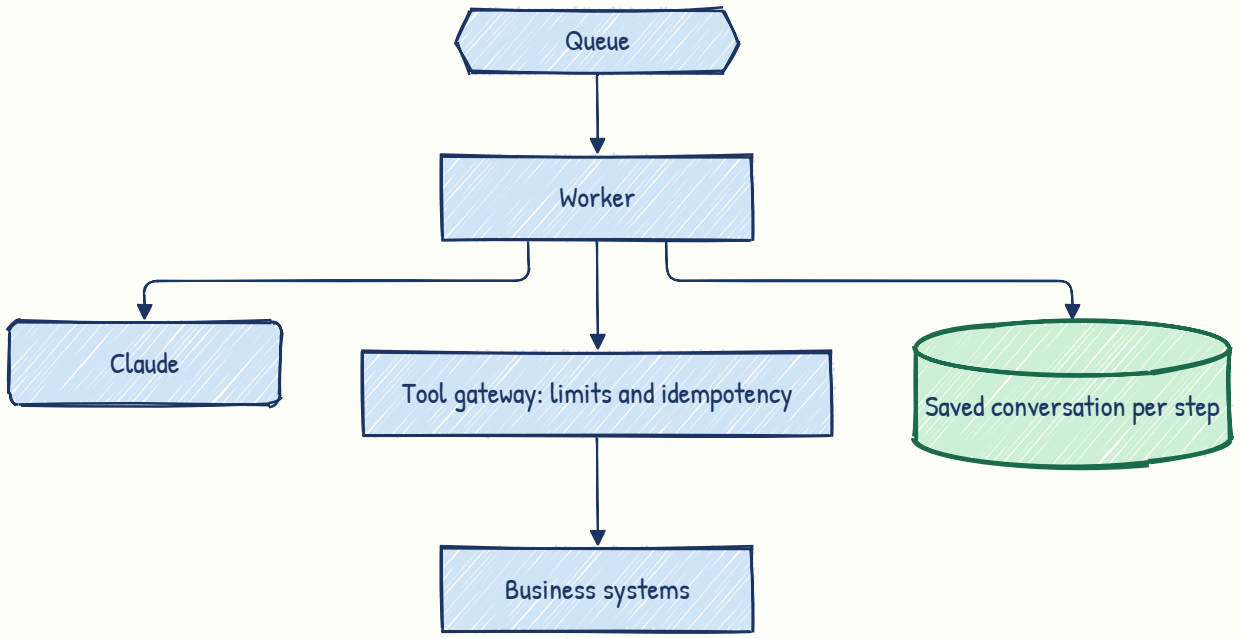


Figure 5. Production. The conversation is saved after each step, and each tool call carries an idempotency key, so a resumed run can't repeat a side effect.

Component	Why it exists	How it communicates
Queue	Decouples runs from requests, and absorbs bursts	Async messages
Worker	Runs the loop, and can stream progress	Consumes jobs, streams to the client
Saved conversation	Resume after a crash	Write after each step
Tool gateway	Validation, timeouts, idempotency, audit	Internal API. The key can be the `tool_use` ID
Limits	Stop runaway spend	Steps, tokens and dollars per run

Failures

Failure	Containment
Claude keeps calling the same tool	Detect repeats, and end with a partial result
Process dies mid-run	Resume from the saved conversation
Tool hangs	Timeout, then return `is_error`
Result too large	Truncate with guidance, or page it (AI06 L3)
Retried step repeats a side effect	Idempotency key
Rate limited (429)	Backoff, respecting `retry-after`

Java

The manual loop on the real SDK, from `ClaudeAgentLoop.java`. It's compiled against `anthropic-java` 3.2.0 and hasn't been run, because no API key is available here.

```
Message reply = client.messages().create(params.build());
StopReason stop = reply.stopReason().orElseThrow();

if (stop.equals(StopReason.END_TURN)) {
    return text(reply);
}
if (stop.equals(StopReason.PAUSE_TURN)) {
    params.addMessage(reply); // resend as is, with no extra user message
    continue;
}
if (!stop.equals(StopReason.TOOL_USE)) {
    throw new IllegalStateException("Cannot continue: " + stop);
}

params.addMessage(reply); // the whole reply, so tool_use blocks stay valid
List<ContentBlockParam> results = new ArrayList<>();
for (ContentBlock block : reply.content()) {
    block.toolUse().ifPresent(call -> results.add(run(call)));
}
params.addMessage(MessageParam.ofUserOfContentBlockParams(results));
```

Line	Why it matters
`reply.stopReason()`	Control flow reads this, and never the text
`addMessage(reply)`	The whole reply, or later `tool_result` blocks have no match
One `addMessage` of `results`	All results go in a single user message

And each failed tool is flagged rather than dropped:

```
return ContentBlockParam.ofToolResult(ToolResultBlockParam.builder()
        .toolUseId(call.id())
        .content(known ? "Order delivered, total $40"
                : "Unknown tool: " + call.name())
        .isError(!known)
        .build());
```

The same agent with the tool runner, from `RunnerAgent.java`, also compiled only. Note that the class is the tool, and the SDK owns the loop:

```
@JsonClassDescription("""
        Look up one order by its ID. Returns status, total and delivery date. \
        Use this for questions about a specific order.""")
static class LookupOrder implements Supplier<String> {
```

Trade-offs

Decision	Gain	Cost
Tool Runner	Less code, built-in loop	Beta dependency
Manual loop	Total control	You own every edge case
Managed Agents	Hosted loop and sandbox	Beta, and files must be uploaded or mounted
Saving per step	Crash recovery	Storage and writes
Parallel calls	Speed	Order isn't guaranteed

When you don't need any of this: if one tool call answers it, a single round trip may do.

Common mistakes

1. **Parsing text for "done".** Use `stop_reason`.
2. **Treating any text as completion.** Replies mix text and tools.
3. **Using a cap as the normal exit.** It should signal failure.
4. **Dropping a failed tool's result.** Each `tool_use` needs a `tool_result`.
5. **Splitting results across messages.** Return them together.
6. **Adding "Continue" after `pause_turn`.** Just resend.
7. **Running tools after `max_tokens`.** Input may be truncated.
8. **No idempotency on retries.** Refunds can repeat.

What to say in an interview

"The loop continues on `tool_use` and ends on `end_turn`. I'd handle `max_tokens`, `pause_turn` and `refusal` explicitly, and never run tools from a truncated turn. A step limit acts as an alarm that reports a partial result, and isn't how we normally finish. For side effects, I'd save state per step, and use the tool call ID as an idempotency key."

Certification lens

How this lesson maps to CCAR-F

TS 1.1. Continue on `tool_use`, finish on `end_turn`, and append tool results each turn. Named anti-patterns: parsing natural language, arbitrary caps as the primary stop, and treating text as a completion signal. The guide doesn't test `pause_turn` or `refusal`, but production needs them.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Customer Support Resolution Agent · Easy. Your loop ends as soon as Claude's reply contains any text. Customers report the agent says "Let me check that" and then stops. What's the fix?

A. Instruct Claude not to write text before calling tools.

B. Continue while `stop_reason` is `tool_use`, and finish on `end_turn`.

C. Wait 2 seconds, and check for further replies.

D. Increase `max_tokens`.

A **B** (TS 1.1). A is a prompt patch on a control flow bug, and text with tool calls is normal. C misunderstands the API. D has nothing to do with this.

Q Q2 · Exam style, scenario: Multi-Agent Research System · Medium. A research agent's loop stops after exactly 5 iterations, returning whatever it has as a finished report. Complex questions get shallow reports. What's the best change?

A. Raise the cap to 50, and keep treating it as completion.

B. Ask Claude to finish within 5 steps.

C. Check whether the report mentions a conclusion before stopping.

D. Finish on `end_turn`, keep a higher limit as a safeguard, and mark any run reaching it as incomplete.

A **D** (TS 1.1). A still reports truncation as success. B reduces depth. C parses natural language.

Q Q3 · Exam style, scenario: Developer Productivity with the Agent SDK · Medium. After Claude calls a tool, your next request returns a 400 saying a `tool_result` has no matching `tool_use`. Your code stores `reply.text` as the assistant message. What's wrong?

A. You saved only the text, so the `tool_use` block is missing. Append the whole assistant message.

B. The tool took too long to respond.

C. The tool's schema is missing a required field.

D. You need `strict: true` on the tool.

A **A** (TS 1.1). B and C would give different symptoms. D affects input validation, not history.

Q Q4 · Exam style, scenario: Customer Support Resolution Agent · Medium. Claude returns three `tool_use` blocks in one reply. Which handling is correct?

A. Run the first, return its result, and wait for Claude to ask again.

B. Return each result in its own user message, in order.

C. Run all three, and return all three `tool_result` blocks in one user message.

D. Run all three, and return only those that succeeded.

A **C** (TS 1.1, and the tool use docs). A discards two requests and makes the next request invalid. B discourages parallel calls. D leaves `tool_use` blocks unanswered.

Q Q5 · Debugging · Hard. A reply has `stop_reason: "max_tokens"` and includes a `tool_use` block for `process_refund` with `{"amount": 12` cut off. The SDK's tolerant parser gives `amount=12`. What should the loop do?

A Not run the tool. The real amount might be 120 or 1250. Check `stop_reason` before running anything, then retry with a higher `max_tokens`, and validate inputs against the schema. Never run a tool from a truncated turn.

Revision - Lesson 2

→ Loop: send, read stop_reason, run tools, append, repeat.

→ Continue on tool_use, and finish on end_turn.

→ Never parse text for completion. Replies can mix text and tools.

→ A limit is a safety net that reports failure, not a normal exit.

→ Append the whole assistant message, including tool_use blocks.

→ All tool results go in one user message, with matching tool_use_id.

→ Return failures with is_error, and never drop them.

→ pause_turn: resend, with no added message.

→ max_tokens: don't run tools. Refusal: stop.

→ Requests grow each turn, so cache the prefix.

→ Tool Runner: SDK loop for your tools. Agent SDK: the Claude Code harness. Managed Agents: hosted.

→ Use idempotency keys, because retries can repeat side effects.

Your notes